

Sports Game Outcome Predictor

Mattia Pecchioli, Luca Serafini

Giugno 2026

1 Introduzione

Sports Game Outcome Predictor è un'applicazione di machine learning che permette di predire i risultati di un evento sportivo calcistico, in particolare, le partite della Serie A italiana.

L'obiettivo è l'implementazione di un **modello di supervised learning** con il seguente scopo: prevedere il risultato di un incontro di Serie A, specificando le percentuali con le quali ogni evento (1-X-2) possa avvenire, una volta appresi dei pattern dai dati passati.

1.1 Strumenti utilizzati

Per lo sviluppo del codice è stato utilizzato Visual Studio Code. Per quanto riguarda il linguaggio, è stato utilizzato Python come da specifiche, nella versione 3.12, con uv come strumento di packet manager. Abbiamo poi organizzato in due diverse directory il lavoro:

- La prima, chiamata **notebook**, è stata utilizzata per la parte di analisi dei dati e di machine learning. In questa parte, come dice il nome stesso, abbiamo preferito lo sviluppo tramite notebook, data la possibilità di spiegare il codice scritto tramite dei commenti in markdown. Un altro motivo è la possibilità di creare delle variabili runtime, che possono essere utilizzate nei diversi blocchi di codice a nostro piacere, senza dover riavviare tutto il codice ad ogni nuova esecuzione;
- La seconda, chiamata **app**, è il programma in sé, al quale è stato associato un file *requirements.txt*, che contiene i soli pacchetti utili al funzionamento dell'applicazione.

L'applicazione viene poi resa fruibile all'utente finale tramite una piattaforma web realizzata in Flask e containerizzata in Docker.

2 Analisi dei dati

L'obiettivo della sezione di analisi dei dati è quello di produrre delle feature derivate dai dataset selezionati (ovvero non presenti nel dataset di riferimento) che possano essere utili per predire i risultati delle partite di nostro interesse. Come anticipato in precedenza, il dominio del nostro modello è formato dalle partite della Serie A italiana, a partire dal campionato 2015/2016. I dataset che sono stati utilizzati sono i seguenti:

- Italian Serie A (football)¹;
- Football data from Transfermarkt².

¹<https://datahub.io/football/italian-serie-a>

²<https://www.kaggle.com/datasets/davidcariboo/player-scores/data>

2.1 Feature Engineering

Nel file *feature-engineering.ipynb* è contenuto il codice relativo a questa parte, che mostra gli snippet di codice che abbiamo realizzato per la produzione delle feature di nostro interesse. I dati da cui siamo partiti, ovvero quelli presenti nel dataset sono i seguenti:

- Squadra di casa e in trasferta;
- Goal di entrambe le squadre;
- Tiri in porta di entrambe le squadre;
- Squadra vincitrice (classe di tre valori: A, H, D);
- Valore di mercato dei singoli giocatori di ogni squadra e associazione dei giocatori ad una partita (o meglio, una semplificazione, che tiene conto dei soli giocatori **titolari**).

Da questi valori, con l'ausilio di librerie come **pandas** e **numpy** abbiamo ottenuto le seguenti feature, che abbiamo ritenuto più indicative per i nostri scopi. Per la precisione, ogni feature³ è stata calcolata per la squadra in casa e per quella in trasferta, ma ai fini del machine learning è stata trasformata in una feature sola, sottraendo il valore di quella in trasferta da quella in casa.

- **WinStreak** (streak di vittorie): un valore intero che indica il numero di vittorie di fila, nello stesso campionato, che sono state fatte nella partita precedente a quella che dovrà essere disputata (ad ogni pareggio o sconfitta questo valore viene azzerato);
- **Rapporto Goal/Tiri in porta**:

$$R_{g,t} = \frac{\sum_i^{\min(5,p)} g[-i]}{\min(5,p)}$$

Questo valore è il rapporto della somma tra i goal nelle ultime (almeno) 5 partite e il numero delle partite in cui sono stati fatti questi goal.

- **HomeAdvantage**: questo parametro è dato dal rapporto dei risultati delle ultime (almeno) 5 partite sul numero delle partite.
- **ELO**: questo valore si ispira al punteggio ELO degli scacchi, che si basa sul fatto di assegnare molti punti per le vittorie con squadre forti, ed assegnarne pochi per vittorie con squadre considerate deboli. Per definizione, abbiamo deciso che inizialmente ogni squadra possiede un ELO di 1500. Definiamo poi con R_H e R_A l'ELO attuale delle squadre in casa e in trasferta. La formula per il calcolo del punteggio atteso è la seguente

$$E_H = \frac{1}{1 + 10^{\frac{R_A - (R_H + H_{adv})}{400}}}$$

per la squadra in casa e

$$E_A = \frac{1}{1 + 10^{\frac{(R_H + H_{adv}) - R_A}{400}}}$$

per quella in trasferta. Definiamo poi il valore S, che vale 1 per la vittoria, 0.5 per il pareggio e 0 per la sconfitta. Infine abbiamo il moltiplicatore dello scarto reti, ovvero il parametro G, che vale 1 nel caso di pareggio o un goal di scarto; 1.5 per 2 goal di scarto e $\frac{11+N}{8}$ per 3 o più goal di scarto. Il valore finale dell'elo della squadra risulterà:

$$\begin{cases} R_{H,new} = R_{H,old} + K \cdot G \cdot (S_H - E_H) \\ R_{A,new} = R_{A,old} + K \cdot G \cdot (S_A - E_A) \end{cases}$$

³Tranne per HomeAdvantage

- **Value:** questo valore indica il valore della rosa della squadra, ovvero la somma del costo dei singoli giocatori **titolari**. L'idea alla base di questa feature è che una rosa che ha un costo maggiore, avrà probabilmente una maggiore resa in campo;
- **PointToMatchRatio:** indica il rapporto tra il numero di punti effettuati ed il numero di partite giocate in precedenza.
- **Z-score:** indica di quante deviazioni standard σ un determinato valore x si discosta dalla media μ del campionato:

$$Z_{score} = \frac{x - \mu}{\sigma}$$

Dal punto di vista algoritmico, questa feature è fondamentale per introdurre dipendenza temporale dell'input rispetto all'output.

2.2 Analisi del dataset

Una volta individuate le caratteristiche, procediamo con l'analisi del dataset. Per prima cosa andiamo a verificare il bilanciamento del dataset. Come si vede dalla figura 1, possiamo notare che il dataset è leggermente sbilanciato. La classe "X", infatti, presenta un numero minore di occorrenze rispetto alle classi "1" e "2".

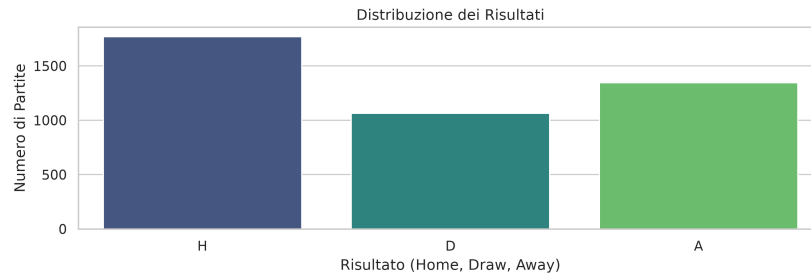


Figure 1: Distribuzione degli esiti all'interno del dataset

Un'altra importante analisi che dobbiamo effettuare è quella della **heatmap delle feature in ingresso**. Il grafico in figura 2 serve a verificare il coefficiente di correlazione tra due feature. Se esso è alto ($> 80\%$) vuol dire che le due feature sono ridondanti se utilizzate assieme, portando al **fenomeno della multicollinearità**.

3 Machine Learning

3.1 Train-Validation-Test

Il dataset complessivo, contenente le partite del campionato di Serie A dal 2015 al 2026, è stato suddiviso in tre sottoinsiemi secondo le seguenti proporzioni:

- **Training Set (70%):** utilizzato per l'addestramento dei modelli per permettere a questi di apprendere le relazioni tra le feature realizzate e gli esiti sul campo.
- **Validation Set (10%):** impiegato per tuning degli iperparametri e per la calibrazione della soglia decisionale sul pareggio.

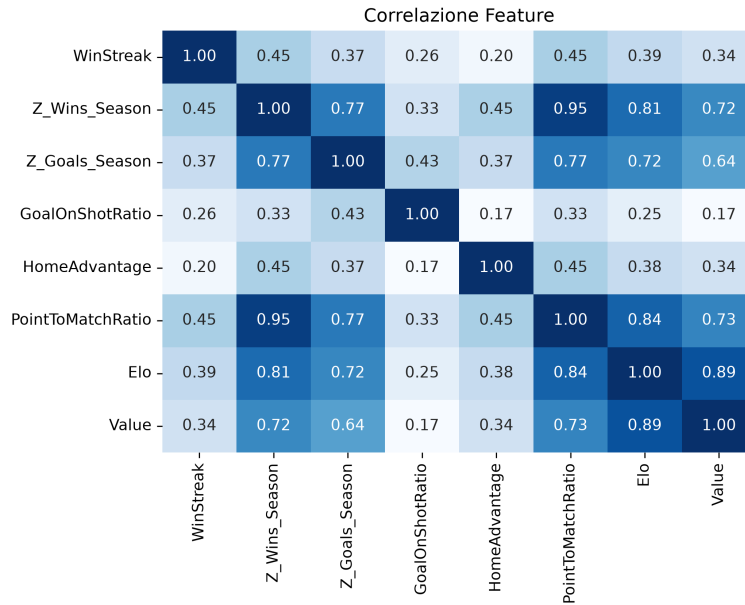


Figure 2: Heatmap di correlazione tra feature

- **Test Set (20%)**: riservato alla valutazione finale e imparziale delle performance dei modelli, garantendo che questi dati non influenzassero in alcun modo la fase di addestramento o di ottimizzazione.

La suddivisione del dataset è stata eseguita secondo un **criterio temporale** poichè nei pronostici sportivi questo è fondamentale per preservare la sequenzialità degli eventi.

3.2 Soglia decisionale sui pareggi

Durante la fase di sviluppo sul validation Set, l'osservazione delle distribuzioni probabilistiche generate tramite la funzione *predict_proba()* ha evidenziato un comportamento critico comune a tutti i modelli testati: la forte tendenza a sottostimare la classe pareggio (X).

Dalla nostra analisi, questo fenomeno è riconducibile principalmente a due fattori:

- **Lo sbilanciamento delle classi nel dataset**: La maggiore frequenza dei segni 1 e 2 spinge i modelli a polarizzarsi verso gli esiti più frequenti nel tentativo di massimizzare l'Accuracy globale, a scapito della classe meno rappresentata, generalizzando però in pessimo modo.
- **La distorsione nell'assegnazione delle probabilità tramite argmax**: Sebbene la probabilità stimata per il pareggio, $P(X)$, si attesti spesso su valori statisticamente rilevanti, i modelli probabilistici tendono comunque a distribuire la massa di probabilità residua in modo tale che il valore massimo venga sistematicamente assegnato ai segni 1 o 2. Di conseguenza, la rigida regola decisionale dell'argmax simmetrico finisce per escludere quasi totalmente il pareggio dai pronostici finali.

Ciò si traduceva in una **recall estremamente bassa** sulla classe X , con il modello che quasi non si esponeva mai sul pareggio.

Per correggere questo problema e consentire al modello di scommettere sul pareggio, abbiamo deciso di utilizzare il validation set per ottimizzare una **soglia decisionale personalizzata**. Se la probabilità stimata per il pareggio supera una determinata soglia (inferiore alla maggioranza assoluta ma statisticamente rilevante per quella classe), il modello assegna il pronostico al segno X . Sul validation set abbiamo cercato la soglia ottima che permettesse di ottenere il miglior F1-macro, cioè la media di tutti gli F1-score. Una volta individuata la soglia ottimale sul validation, questa è stata applicata sul test Set per la valutazione finale del progetto.

Abbiamo preferito utilizzare la soglia personalizzata anziché il parametro *class_weight = balanced* presente in alcuni modelli, per evitare di influenzare artificialmente le probabilità generate da questi.

3.3 Random baseline

Prima di procedere con l'addestramento dei modelli di Machine Learning complessi, è stata realizzata una random baseline di riferimento, con lo scopo di stabilire la **soglia di performance minima** che qualunque modello predittivo deve tassativamente superare per dimostrare una reale capacità di generalizzazione e apprendimento dalle feature.

La baseline è stata implementata assegnando l'esito di ogni match del test set simulando il **lancio di un dado a tre facce equiprobabili**.

L'introduzione di questa baseline evidenzia il valore aggiunto del nostro approccio. Senza un benchmark casuale, l'ottenimento di un'accuracy (ad esempio) del 45% o 50% potrebbe sembrare modesto in senso assoluto; sapendo però che il punto di partenza stocastico è il 33.3%, tale incremento dimostra che le feature estratte e i successivi modelli di Machine Learning estraggono pattern predittivi reali e significativi.

3.4 Regressione logistica

Il modello sul quale ci siamo concentrati per gran parte del progetto è la **Regressione Logistica Multinomiale**, nota anche come **Softmax Regression**.

Essa estende la regressione logistica binaria mappando l'input su uno spazio di probabilità a tre dimensioni, garantendo che la somma delle probabilità predette per i tre segni sia sempre pari a 1.

3.4.1 Formulazione matematica

Dato un vettore di feature in ingresso $\mathbf{x} \in R^d$, il modello calcola innanzitutto uno score $s_k(\mathbf{x})$ per ciascuna delle K classi mediante una combinazione lineare dei pesi del modello $\boldsymbol{\theta}^{(k)}$:

$$s_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)}$$

Per convertire questi score in probabilità, viene applicata la funzione **Softmax** (σ). La probabilità che l'istanza $\mathbf{x}$ appartenga alla classe k (con $k \in \{1, X, 2\}$) è definita come:

$$p(y = k | \mathbf{x}) = \sigma(s(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

3.4.2 Funzione di Costo: Cross-Entropy

Durante la fase di addestramento sul Training Set, il modello minimizza una funzione di costo chiamata **Entropia Incrociata (Cross-Entropy Loss)**. La formula del costo medio complessivo $J(\boldsymbol{\Theta})$ è definita come:

$$J(\Theta) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_k^{(i)} \log(\hat{p}_k^{(i)})$$

Dove:

- n è il numero totale di partite nel set di addestramento;
- $y_k^{(i)}$ valore binario $\{0, 1\}$ che indica se la partita i -esima è terminata effettivamente con l'esito k ;
- $\hat{p}_k^{(i)}$ è la probabilità stimata dal modello Softmax per la classe k sulla partita i -esima.

L'ottimizzazione **minimizza la distanza tra le distribuzioni di probabilità reali e quelle stimate**, punendo duramente le predizioni errate formulate con alta confidenza dal modello.

3.4.3 L'Argmax

Nei classificatori multinomiali, la predizione avviene applicando la regola dell'argmax:

$$\hat{y} = \arg \max_{k \in \{1, X, 2\}} \hat{p}_k$$

Tuttavia, come discusso nella sezione relativa alla calibrazione del modello, abbiamo deciso di utilizzare una soglia personalizzata anziché tale funzione.

3.5 Metriche di valutazione

Le metriche di valutazione utilizzate per analizzare la validità dei vari modelli e per confrontare le performance predittive ottenute sul test set sono:

- **Accuracy:** rappresenta la frazione di pronostici corretti rispetto al totale delle partite nel test set. È la metrica macroscopica fondamentale utilizzata per validare il superamento del benchmark fissato dalla Random Baseline:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}}$$

L'accuracy è però un dato parziale in presenza di classi sbilanciate (come la maggiore frequenza storica delle vittorie in casa). Per tale motivo, viene integrata con metriche specifiche per classe.

- **Precision:** esprime la percentuale di predizioni corrette rispetto al totale dei pronostici emessi dal modello per una determinata classe. Per una generica classe c , la metrica è definita come:

$$\text{Precision}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c}$$

- **Recall:** quantifica quante delle partite effettivamente terminate con il segno c siano state identificate correttamente dal modello:

$$\text{Recall}_c = \frac{\text{TP}_c}{\text{TP}_c + \text{FN}_c}$$

- **F1-Score:** rappresenta la media armonica tra Precision e Recall, offrendo un singolo parametro per valutare la stabilità e l'equilibrio complessivo del classificatore su ciascun esito. Tra tutte le metriche è il **parametro che abbiamo cercato di massimizzare**, per fare in modo che il nostro modello generalizzasse il più possibile.

$$\text{F1-Score}_c = 2 \times \frac{\text{Precision}_c \times \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

Questa metrica si rivela di fondamentale importanza per l'analisi della classe Pareggio (X). Essendo questo l'esito più complesso da prevedere a causa della natura intrinsecamente stocastica del gioco del calcio, l'F1-Score **permette di verificare che il modello mantenga una reale stabilità predittiva su tutte e tre le classi**, evitando che l'algoritmo ottimizzi l'Accuracy globale predicendo esclusivamente vittorie in casa o vittorie in trasferta.

Trattandosi di un problema di classificazione multiclasse con tre classi mutuamente esclusive (1, X , 2), il calcolo di tali indici segue la logica **One-vs-Rest**, in cui ciascuna classe viene contrapposta ciclicamente alle rimanenti due, considerate come un'unica macro-classe negativa.

4 Bet Simulator

Il progetto si conclude con la creazione di una sezione di **bet simulation** (in `backtest.ipynb`), che dà la possibilità di ottenere il possibile guadagno ottenuto applicando i nostri modelli in un contesto reale (i campionati del test-set), puntando sulla classe (vittoria in casa, pareggio o vittoria in trasferta) che viene predetta come favorita. Naturalmente quest'ultima parte può essere testata solamente sui campionati del **testset** (2024-2025, 2025-2026), di cui conosciamo i risultati e sui quali il modello non è stato allenato e validato.

L'obiettivo di quest'ultima parte è quello di effettuare dei confronti tra i vari modelli che abbiamo prodotto e la random baseline, per capire concretamente l'effettiva differenza con una scelta casuale dei risultati delle partite. Per semplicità non utilizzeremo una tecnica di scommessa complessa, bensì scommetteremo un valore fisso (10 euro) per ogni partita.

4.1 Baseline Casuale

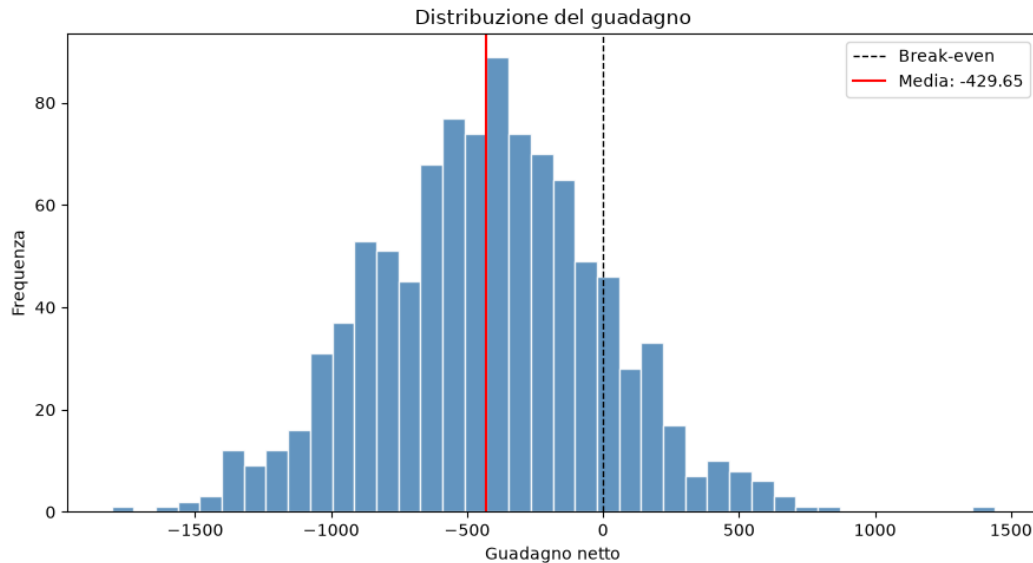


Figure 3: Distribuzione backtest casuale

In questo caso diamo ad ogni evento la probabilità del 33% di accadere. Ci aspettiamo quindi un numero di perdite superiori a quelle delle vincite. Naturalmente, in questo caso, data la grande varianza di questo tipo di scommesse, abbiamo la necessità di effettuare una serie di prove ripetute (utilizziamo

il **Metodo Monte Carlo**⁴) sul test-set. La distribuzione dei risultati si avvicina ad una **gaussiana**, con una media approssimativa di -500 euro (Figura 3). Come si poteva prevedere, questo metodo di scommessa è controproducente.

4.2 Confronto tra modelli

Una volta analizzato il comportamento della baseline randomica, possiamo utilizzare i tre modelli che abbiamo sviluppato per effettuare il backtest. I risultati ottenuti sono i seguenti:

- **Regressione lineare**: otteniamo un modesto valore di surplus, di circa 330 euro;
- **XgDBoost**: il risultato di XgBoost è superiore al precedente di circa 75 euro (414), notiamo quindi un buon aumento del capitale finale;
- **Random Forest**: è il modello più profittevole, ottenendo un valore doppio rispetto alla regressione logistica, arrivando ad un profitto di 780 euro.

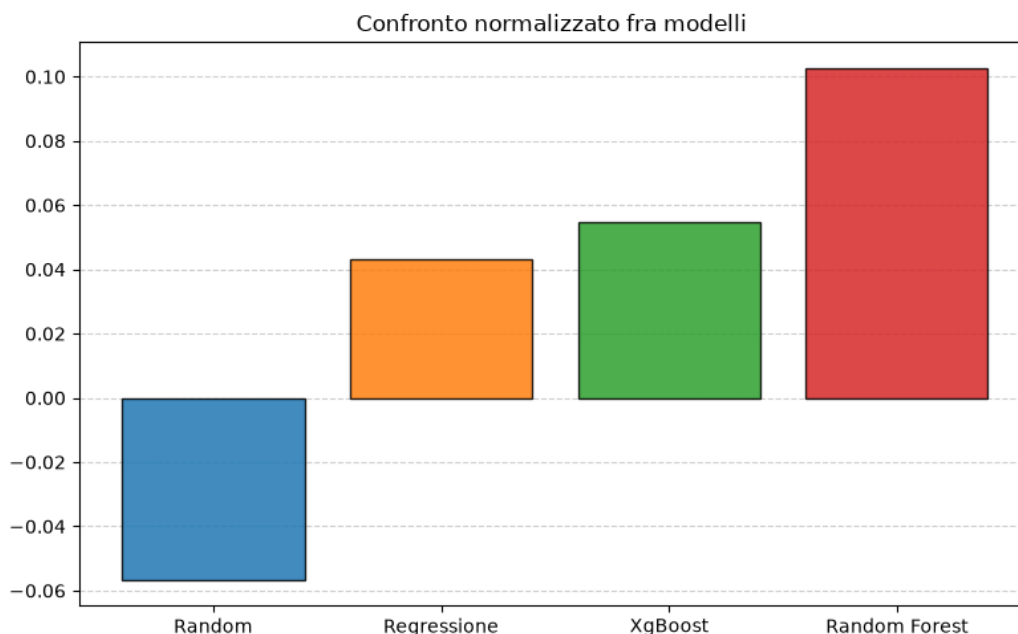


Figure 4: Distribuzione backtest (normalizzata)

In ultima analisi è stato creato un grafico per il confronto tra i guadagni dei modelli, normalizzati rispetto alla cifra investita (7600 euro, date le 760 partite giocate nei due campionati). Possiamo notare che solo il primo modello va in negativo, erodendo quasi del 6% il capitale investito. I due modelli intermedi, portano invece ad un guadagno compreso tra il 4 e il 6%, risultato più che apprezzabile. Con oltre il 10% di guadagno, il modello random forest si classifica come il migliore, con netto distacco dai precedenti.

5 Difficoltà incontrate

Nello svolgimento del progetto sono state incontrate diverse difficoltà, di cui spiegheremo le principali, ovvero:

⁴https://it.wikipedia.org/wiki/Metodo_Monte_Carlo

- Comprendere le feature corrette per massimizzare le performance dei modelli;
- Difficoltà nella predizione del pareggio.

5.1 Scelta delle feature

Il primo problema che ci si è presentato è stato quello della scelta delle feature. Abbiamo infatti notato che, provando ad inserirle casualmente, il modello variava di molto le metriche in output, ed era quindi necessario inserirle con un senso logico. Per ovviare a questo problema abbiamo analizzato questo fenomeno e concluso che:

- **Più feature non significa più accuratezza:** si è notato che l'utilizzo di un numero elevato di feature non è la soluzione ideale per migliorare il modello. Anzi, in alcuni casi, sono sufficienti solo pochi attributi per far performare al meglio il modello, aggiungendone anche solo una in più, il risultato era peggiorativo;
- **Alcune feature sono ridondanti:** con la realizzazione della heatmap della correlazione, abbiamo notato che le feature con un certo intervallo di correlazione (valutato empiricamente oltre l'80%), se utilizzate assieme, portavano ad un grosso peggioramento di performance.

A seguito di queste conclusioni, siamo riusciti a scegliere le feature corrette per ogni modello, per ottenere i migliori risultati.

5.2 Pareggio

La maggior problematica che abbiamo incontrato è stata quella della previsione del pareggio. Inizialmente tutti i nostri modelli presentavano percentuali molto basse (prossime allo 0) nella recall e nella precision di questa classe. Abbiamo quindi analizzato i risultati dell'inferenza dei modelli sul test-set, notando che la percentuale della classe pareggio presentava dei valori abbastanza alti (anche oltre il 30%), ma che non superavano mai quello della vittoria in casa e/o in trasferta, facendo sì che il modello non predicesse mai questa classe. Abbiamo quindi deciso di cambiare strategia: al posto di predire il risultato di una partita con la classe che presentasse la maggior percentuale, abbiamo implementato una soglia per i pareggi, oltre la quale l'omonima classe venisse automaticamente scelta come risultato. Questa soglia è stata valutata utilizzando il validation-set per scegliere il suo valore ottimale. I risultati ottenuti sono stati molto buoni: la precision e la recall sono incrementate, arrivando a sfiorare il 40%.

6 Implementazioni future

Nonostante i risultati ottenuti si rivelino soddisfacenti e superiori alla baseline di riferimento, siamo consapevoli del fatto che il nostro progetto presenti alcune limitazioni e si presti a diverse considerazioni che potrebbero essere esplorate in versioni future.

6.1 Modelli

Una delle principali limitazioni del nostro approccio risiede nell'aver trattato il problema come una classificazione multiclasse diretta basata su feature tabulari aggregate. Informandoci abbiamo notato che l'approccio standard per la modellizzazione degli eventi calcistici prevede spesso l'utilizzo di modelli statistici specifici, tra cui spicca il **modello di Dixon-Coles**, il quale però non avendolo trattato abbiamo preferito non usarlo.

6.2 Combinazione delle feature

Un ulteriore margine di miglioramento riguarda la modalità di combinazione delle feature. Attualmente, le variabili descrivono lo stato di forma e le performance passate tramite medie mobili e formule matematiche. Tuttavia, feature o combinazioni diverse delle stesse, potrebbero consentire ai modelli di ottenere performance ancora migliori.